Northern Illinois University

CSCI 490-B9 – Deep Learning

Course Project, Part 1: The Proposal

Fall 2026

Released: Tuesday, October 27, 2026

Due: your choice of task: Thursday, October 29, 2026, 11:59pm
the proposal: Thursday, November 5, 2026, 11:59pm

Weight: the proposal is 5% of your course grade; the whole project is 35%

Submit: on Blackboard: the link to one commit of your GitHub repository (Appendix A.6)

Work: individual (see *Working together* below)

The project at a glance

The course project takes the place of the final exam. In five weeks you take one real task, train and evaluate models for it yourself, and explain what you found. Your project lives in a GitHub repository that you update as you work: it is your lab notebook during the semester and a piece of your portfolio afterwards.

Deliverable	Weight	Due	It shows that
Proposal	5%	Thu 11/5, 11:59pm	your repository has the task, a data check, a Level 0 result and a plan
Progress report	5%	Thu 11/19, 11:59pm	Level 1 is trained and evaluated
Presentation	10%	in class, Tue 12/1 or Thu 12/3	you can explain your results
Final report	15%	finals week, date on Blackboard	the full study, in IEEE format, with the repository
Total	**35%**		

How the project is graded. *A careful project with a negative result is worth more than a sloppy project with a good number.* You are graded on technical correctness, completeness, experimental rigor and clarity, not on state-of-the-art accuracy. You lose points for a test set used for tuning, a missing baseline, or a number you cannot show in a results file; you do not lose points because your model did not beat a paper.

Four levels. Every project, whichever task, climbs the same four levels. They follow the lectures, so each level uses what the class has just covered.

Level	What	Example: Task A	Due with
0	a classic baseline	TF-IDF and logistic regression (given)	proposal, 11/5
1	a neural model you train	an MLP, or an LSTM on word embeddings	progress report, 11/19
2	a pretrained Transformer, fine-tuned	DistilBERT or SciBERT	presentation, 12/1
3	one open question, answered with evidence	does a model trained on 2020 papers still work in 2025?	final report

Levels 0–2, done carefully, make a complete project. Level 3 is where a project stands out, and it has no fixed answer.

Choosing a task: two ways in

1. **A set task (recommended).** Choose Task A, B or C below. All three use MustCite-Mini, a dataset prepared for this course. The data, the split, the evaluation and a Level 0 baseline are ready in the starter repository. Everyone can finish a set task, and there is room at the top for anyone who goes further.
2. **Your own task, or other data, with my approval.** See *Your own task* below. It must be at least as hard as the set tasks.

The data: MustCite-Mini

MustCite (`https://mustcite.com`) is a search engine built in my lab on about 180,000 papers from top AI/ML venues, with their references matched to each other. MustCite-Mini is a subset of it: 18,000 papers from six venues (NeurIPS, ICML, ICLR, CVPR, ACL, AAAI) and six years (2020–2025), 500 papers per venue and year. It is in the starter repository (`data/`) and as a zip on Blackboard; `data/README.md` describes every column.

File	What is in it	Rows
`papers.csv.gz`	paper id, title, abstract, venue, year, split	18,000
`citations.csv.gz`	one row per citation, both papers in the set	56,033
`contexts.csv.gz`	the sentences around a citation in the full text, labeled core or background	52,072

The split is fixed, by year: train on 2020–2023, choose on 2024 (validation), test on 2025. The test papers are newer than anything seen in training; 2025 has topics that 2020 barely had. A model has to work forward in time, which is the realistic setting. The functions in `mustcite.py` load each task with this split and score predictions; everyone uses the same functions, so everyone's numbers mean the same thing.

Task A: Which conference is this paper from? (the default)

Input	the title and abstract of a paper
Output	one of the six venues
Data	`load_task_a()`: 12,000 training, 3,000 validation, 3,000 test papers, 500 per venue and year
Metric	`evaluate_a()`: accuracy, macro-F1 and the confusion matrix
Level 0	TF-IDF and logistic regression: **42.4%** validation, **39.2%** test accuracy (guessing: 16.7%)

What makes it hard. ACL (language) and CVPR (vision) are easy to recognize. NeurIPS, ICML and ICLR publish the same kind of machine learning paper, and AAAI publishes a bit of everything. In the Level 0 confusion matrix most ICLR papers are predicted as NeurIPS.

Level 3 ideas. Can any model tell NeurIPS from ICLR, or is the limit in the data? Which words give a venue away, and are they about the topic or about the writing style? How much worse is a model on 2025 than on 2024, and which topics are new? Does the title alone carry as much as the abstract? Is a zero-shot LLM better or worse than your fine-tuned model, and at what cost?

Task B: Is this citation core, or background?

Input	up to three sentences around a citation, with their section names; the cited paper is `[CITATION]`, other citations are `[REF]`
Output	1 if the cited work is a core citation, 0 if it is background
Data	`load_task_b()`: 34,441 training, 8,760 validation, 8,871 test contexts; 18% are core
Metric	`evaluate_b()`: precision, recall and F1 of the core class, and ROC-AUC
Level 0	TF-IDF and logistic regression: F1 **0.711** validation, **0.723** test; ROC-AUC 0.93

Two examples. *Core (1)*, from a section "5. Experiments": "CelebA [CITATION] is a large-scale image dataset of celebrity faces." *Background (0)*, from "1. Introduction": "Transformers [CITATION] have achieved remarkable success across fields."

The labels were made by an LLM, not by people. It was asked whether the cited work is a baseline or a direct comparison in the citing paper, or the source of its main dataset, benchmark, task or method (1), or only background or related work (0). Some labels are wrong. Accuracy is the wrong metric here: predicting 0 for everything is 82% accurate.

Level 3 ideas. Label 100 test contexts yourself: how often is the LLM wrong, and is your model closer to you or to the LLM? How much does the section name alone predict? Does a model trained on vision papers work on language papers? Can a small model you train match the LLM, and how much cheaper is it?

Task C: Which papers will a 2025 paper cite?

Input	the title and abstract of a new paper, and the citation graph of the earlier papers
Output	the earlier papers, ranked from most to least likely to be cited
Data	`load_task_c("val")` and `load_task_c("test")`: the *queries* (the 2024 or 2025 papers), the *candidates* (all earlier papers, 12,000 or 15,000) and the *known edges* (the citations made by the candidates)
Metric	`evaluate_c()`: recall@10 and MRR (mean reciprocal rank)
Level 0	rank by TF-IDF similarity of the abstracts: recall@10 **0.265** validation, **0.235** test; MRR 0.40 and 0.35

This is the hardest of the three, for those who liked the GNN part of the course. Level 0 reads the text only. Level 1 is a GNN for link prediction on the known edges (for example GraphSAGE with TF-IDF features reduced by SVD), and Level 2 uses node features from a pretrained Transformer. To train, use the known edges: hide some of them and learn to recover them.

Level 3 ideas. Does knowing who cites whom beat reading the abstract, and for which papers? Are highly cited papers too easy to predict, and what happens if you remove them? Do papers cite across venues, and can your model find those citations?

Your own task (needs my approval)

You may propose another task on MustCite-Mini or a task on other data. I approve it or ask for changes.

Approved	**Sent back**
A model you train on a real task, evaluated honestly	Calling an API and reporting what it says
A pretrained model fine-tuned on your task	Re-running someone's notebook
Any domain: images, sound, text, graphs, science	Tutorial data: MNIST, Iris, Titanic, IMDb and the like
Data in hand; one training run under 30 minutes on a free Colab T4	A task where a simple baseline already gets 95%

The test: your baseline is far from perfect, and you can say why a better model should help.

To request it, post on Blackboard by Thursday, October 29, 2026, 11:59pm: (1) the task, as input and output; (2) a link to the data, its size and license, and the number of examples per class; (3) how you will split it, and why; (4) your baseline and, if you have run it, its number; (5) your plan for Levels 1–3. I answer by Saturday, October 31. If your task is not approved by Tuesday 11/3, you do Task A.

What to do: seven steps

1. **Post your choice** on Blackboard by Thursday, October 29, 2026, 11:59pm: Task A, B or C, or a request for your own task (above). One line is enough for a set task.

2. **Create your repository from the starter repository** and clone it to your laptop (Appendix A.1–A.4). The starter repository is at `https://github.com/zhangleiniu/csci490-project-starter`.

3. **Run Level 0.** In the repository folder, run `python level0_task_a.py` (or `_b`, `_c`). It takes under a minute on a laptop and writes two files to `results/`. Commit them (Appendix A.5). Own task: write and run your own Level 0 script, saving its metrics with `save_results()` or the same JSON format.

4. **Check the data.** Write `data_check.py`, with your agent if you use one:

```
Write data_check.py for Task [A, B or C]: load the data with the functions in
mustcite.py, print the number of examples per class and per split, and write
samples.md with three examples of each class.  Run it.
```

Then read `samples.md` yourself: do the labels make sense? Does every class appear in every split? Commit both files. (Task C has no classes: count queries, candidates and edges per split.)

5. **Fill in `AGENTS.md`.** Write your task on its first line and keep the rules. A coding agent such as opencode reads this file at the start of every session, so it will not re-split the data, report the wrong metric or tune on the test set behind your back.
6. **Write the README** (next section). It is your proposal.
7. **Commit, push and submit** the link to your last commit before the deadline (Appendix A.6).

The README: six questions

The starter repository has a `README.md` with these headings. Fill in every part.

1. **Task.** What goes into the model, and what comes out? Give one real example from the data with its label.
2. **Motivation.** Who would use it, and what makes it hard? Three to five sentences.
3. **Data.** Where the data comes from and how it is split. The table from `data_check.py` (examples per class and per split), a link to `samples.md`, and one thing you noticed when you read the samples. Own data: also its size and license.
4. **Method.** Your Level 0 result on validation and test, copied from `results/`. Then the models you plan for Levels 1 and 2, and for each the number you expect on validation. Your expected numbers may turn out wrong; the final report compares them with what you got, and that comparison is part of a good discussion. A good plan has a baseline, a main model and one comparison: for example, in Task A, Level 0, a fine-tuned DistilBERT, and an LSTM or the title alone against title and abstract.
5. **Evaluation.** Which metric, which split, and what counts as success. Say why the metric fits:

Kind of task	**Report**
balanced classes (Task A)	accuracy, macro-F1, confusion matrix
a rare positive class (Task B)	precision, recall and F1 of that class, ROC-AUC
ranking (Task C)	recall@K and MRR
regression, forecasting (own task)	MAE or RMSE, next to a naive guess

6. **Plan.** What you will do each week until 12/1, and your Level 3 question: one question with no fixed answer, and the evidence that would answer it.

At the top of the README: your name and a note on AI tool use, saying which tools you used and for what.

Rules

1. **The split is fixed and the test set is used once.** Choose models, hyperparameters, epochs and thresholds on the validation set. Evaluate each final model once on the test set. Never re-split the data or train on validation or test rows. Related samples stay together (for own data: one person, patient or recording per split).

2. **Every number comes from a file.** Save the metrics of every run with `save_results()` and commit the files in `results/`. A number in your README that is not in a results file counts as made up.
3. **Do not edit** `mustcite.py` or the files in `data/`.
4. **Size.** One training run must take less than 30 minutes on a free Colab T4 GPU. Too big? Cut the data, the text length or the model; never cut the evaluation.
5. **Models and libraries.** Any PyTorch-based library is allowed (Hugging Face Transformers, PyTorch Geometric, scikit-learn). Unlike Assignment 1, pretrained models are allowed and Level 2 needs one. An LLM API may be used as a comparison, never as your model.
6. **Commit as you work**, with messages that say what changed. Do not commit model weights.

Coding agents are allowed, and the rules of the agent lab still apply: say which tool you used and for what; keep the split, the metric and "test once" in `AGENTS.md`; every number comes from a log you can show; and you can explain any line of your code, because you will be asked, in the presentation or in office hours. The agent writes code fast. Choosing the model, the comparison and the Level 3 question is still your job, and that is what is graded.

Grading of the proposal (5% of the course grade)

I grade the commit you link to, not the later state of your repository.

Part	Share	Full marks when
Task and motivation	20%	the input, the output and one real example are clear; the motivation says what makes it hard
Data loaded and checked	30%	`data_check.py` runs; the table and `samples.md` are committed; the README says what you saw
Level 0 result and model plan	20%	the Level 0 numbers match `results/`; Levels 1 and 2 are named, each with an expected number
Evaluation plan	20%	the metric fits the task and the reason is given; validation for choices, test once
Weekly plan	10%	week by week until 12/1, with a concrete Level 3 question

You get written feedback within a week. The plan can change after feedback; that is what it is for.

Submission checklist

1. The README has your name, the AI tool note, and all six parts filled in; no `<angle brackets>` left.
2. `AGENTS.md` names your task.
3. `results/` has the two Level 0 files, and the README's Level 0 numbers match them.
4. `data_check.py` and `samples.md` are committed.
5. Everything is pushed: the files you see on GitHub are the files you mean to submit.
6. The repository is public, or private with `zhangleiniu` added as a collaborator (Appendix A.2).

7. On Blackboard, paste the link to your last commit (Appendix A.6).

Late policy (from the syllabus): 10% of the points you received per day or portion of a day late (weekends count); no submissions accepted more than 3 days late. The time of a submission is the time of the linked commit and of the Blackboard post, whichever is later.

After the proposal. Progress report (11/19): Level 1 committed and in the README's results table. Presentation (12/1 or 12/3): Levels 0–2. Final report (finals week): IEEE format, all levels and the Level 3 question. Each comes with its own short handout.

Working together, and academic integrity

The project is **individual**: your code, your experiments and your writing are your own. Working with classmates on GitHub is welcome and is how you learn to collaborate on code, but it is not graded: you may open issues on each other's repositories, review each other's code, and send pull requests that fix a bug or improve the documentation. A pull request must not add a model or an experiment to someone else's project. Code you take from anywhere, including a classmate's repository or a tutorial, must be credited in a comment. Copying a classmate's models, results or text is academic misconduct and will be handled as described in the syllabus.

There is no TA. Questions come to me in class, in office hours (Tue/Thu 3:15–4:45pm) or by email, early.

Appendix A: Git and GitHub, step by step

A.1 What you need. A GitHub account (`https://github.com/signup`); Git (on a Mac, run `git --version` in the Terminal and accept the install; on Windows, install it from `https://git-scm.com`); and VS Code, which you used in the agent lab.

A.2 Create your repository from the template.

1. Open `https://github.com/zhangleiniu/csci490-project-starter` and click *Use this template → Create a new repository.*
2. Owner: you. Name: for example `csci490-project`. Choose *Public* (recommended: it becomes part of your portfolio) or *Private*. Click *Create repository.*
3. If private: *Settings → Collaborators → Add people →* `zhangleiniu`. I cannot grade a repository I cannot open.

A.3 Clone it to your laptop. On your repository's page, click the green *Code* button and copy the HTTPS address. In VS Code: *View → Command Palette → Git: Clone*, paste the address, choose a folder, and open it. (In a terminal: `git clone <address>`.) The first push asks you to sign in to GitHub; follow the browser window.

A.4 Install the packages. In the VS Code terminal, inside the repository folder:

```
python3 -m venv .venv
source .venv/bin/activate                          (Windows: .venv\Scripts\activate)
pip install numpy pandas scikit-learn matplotlib
```

That is enough for Level 0 and the data check. Train Levels 1 and 2 on Colab (Appendix B), where PyTorch and Transformers are already installed.

A.5 Commit and push. A *commit* is a saved snapshot of your files with a message; *push* sends your commits to GitHub. In VS Code, open *Source Control* (the branch icon on the left), click + next to the files to include, type a message, click *Commit*, then *Sync Changes*. In a terminal:

```
git add level0_task_a.py results/
git commit -m "Run Level 0 for Task A: 42.4% validation accuracy"
git push
```

Commit after each step that works. A good message says what changed, not "update". Your history then shows how the project grew, which is what I look at if I have a question about your work.

A.6 The link to one commit. On your repository's page on GitHub, click the clock icon with *N commits*, click the commit you want to submit, then *Browse files*. Copy the address from the browser: it looks like `https://github.com/<you>/<repo>/tree/<40 letters and digits>`. That page shows the repository exactly as it was at that commit. Paste it on Blackboard. If your repository is public, check the link in a private browser window.

A.7 Working together (optional, not graded). On a classmate's repository: *Issues → New issue* to ask a question or point out a bug; or *Fork*, change a file in your copy, and open a *Pull request* to propose the change. The owner decides whether to merge it.

Appendix B: Running your code on Colab

Write and commit code on your laptop; run the long training on Colab's free GPU.

1. Open `https://colab.research.google.com`, *File* → *Open notebook* → *GitHub*, paste your repository's address and open `run_on_colab.ipynb`.
2. *Runtime* → *Change runtime type* → *T4 GPU*.
3. In the first cell, set `REPO` to your repository's address. Run the cells from top to bottom: they clone your repository, install the packages, check the GPU, run a script and download `results.zip`.
4. Unzip `results.zip` into `results/` on your laptop and commit the new files.

Colab deletes its files when the session ends, so download the results as soon as a run finishes. Changed your code? Commit and push it from your laptop, then run the notebook again: it clones the latest version.

A private repository needs a token to clone. On GitHub: *Settings* → *Developer settings* → *Personal access tokens* → *Fine-grained tokens* → *Generate new token*; give access to this repository only, with *Contents: Read-only*. In Colab, replace the clone cell with

```
from getpass import getpass
token = getpass("GitHub token:  ")
!git clone -q https://$token@github.com/<you>/<repo>.git project
%cd project
```

Never paste the token into a cell as text and never commit it: anyone who has it can read your repository.